MY EXTRA NOTES

Fundamental Concepts of PINNs

Unlike traditional artificial neural networks that rely on standard input (X) and output (Y) datasets, PINNs use datasets derived directly from physics, specifically governing differential systems and boundary conditions.

The architecture includes an input layer, hidden layers with neurons, and an output layer that generates a scalar prediction.

Forward propagation is used to generate a prediction, but it does not perform the actual learning.

Backpropagation facilitates the actual learning process, which occurs after the residual is found.

The residual is calculated as the difference between the network's prediction and the underlying physics equation.

Collocation points are used to evaluate predictions at random locations within the domain.

An epoch represents a complete training round or iteration for the network.

Time Models and Optimization

PINNs can model continuous-time systems, such as the Burgers and Schrödinger equations.

They can also model discrete-time systems using techniques like Runge-Kutta integration and the Allen-Cahn equation.

The loss function evaluates both standard predictions and physical rules to ensure they do not contradict one another.

The Adam optimizer is utilized to diminish the total loss function.

A fixed learning rate scheduler heavily dictates network performance: a large learning rate corrects errors with large steps and takes less time, but it can cause the optimization to bounce or skip.

Conversely, a small learning rate causes the network to learn meticulously through microscopic steps.

Training Challenges and Trade-Offs

One of the negative aspects of PINNs is that training is notoriously difficult due to stiff optimization landscapes.

The training process often requires gradually increasing the error margin and teaching the data to the network slowly.

Because PINNs rely on soft constraints rather than hard constraints, the model's outputs are not always 100% correct.

The system can train itself at virtual points without strictly penalizing the model, which is used for predicting phenomena like sound waves.

While the training phase is long and costly, the inference phase is fast and fluid.

Extending PINNs

Fractional PINNs: In this extension, standard derivative parts are handled by automatic differentiation, while fractional and integral parts are calculated using traditional numerical methods.

Delta PINNs: These are designed for complex and irregular geometries by utilizing the internal functions of the Laplace-Beltrami operator as an external coordinate system.

Curriculum Regularization: This training strategy involves starting with a physics penalty of zero and gradually increasing the physics rules as the network grasps the underlying data.

Pareto Fronts: Hyperparameters must be carefully adjusted to establish a balance between data loss and physics loss; as this balance is modified, Pareto fronts emerge.

Advanced Applications (PML and Interfaces)

Split Field PML (Perfectly Matched Layers): This technique is applied to minimize fake reflections and to explicitly limit the computational domain.

A combined loss function is implemented by enforcing the governing equation throughout the entire PML region.

Interface Conditions: The network achieves a continuous approximation of interface conditions by utilizing a Heaviside function.

Advanced network evaluations rely on Neural Tangent Kernel (NTK) analysis and first-order formulations, such as first-order Maxwell equations.

Overlapping domains are leveraged as an effective domain decomposition strategy.

SUMMARY OF OUR PAPERS

	XPINNs	XI-PINN
Number of Networks	Separate network per subdomain	Single network for all domains
Interface Type	Static	Moving, time-dependent
How Interface is Represented	Residual of sub-networks + average solution matching	Extra input variable (level-set/extended variable)
Parameter Sharing	None (each network independent)	Yes (single network, shared parameters)
Main Motivation	Parallelization, large/complex geometries	Moving interfaces, reducing spectral bias

1. XPINNs (Jagtap & Karniadakis, 2021)

Description: Solves a PDE problem by dividing the entire domain into subdomains, using a separate network for each subdomain + combining them with an average to create a global solution.

✅ Pros: Can be applied to any PDE; parallelizable; can compute solutions in large/complex geometries.

❌ Cons: If the interface between subdomains is not chosen correctly, it can create an artificial interface (depth, activity, etc.).

2. PINNs for EM Interfaces (Nohra & Dufour, 2024)

Description: Models Maxwell's equations for dielectric/magnetic materials; solves for a continuous field across the interface (level-set method) and provides an accurate analytical solution.

✅ Pros: Provides an accurate analytical solution for interfaces with different material properties; also prevents oscillations in the solution (dielectric) (good for inverse problems).

❌ Cons: Smoothing approach can be slightly weak on sharp interfaces, can cause minor errors; does not fully test the contrast problem.

3. AdaI-PINNs (Roy, Annavarapu, Roy, Sarma, 2024)

Description: Reduces the solution space by adding an additional activation term to the PINN loss, eliminating the need for additional equations.

✅ Pros: Provides 2-6x more accuracy than standard PINNs, requires fewer epochs, works with less data, adds fewer extra parameters.

❌ Cons: Only tested on steady-state (static) problems; not applicable to high-contrast (1-Ox) or transient problems.

4. XI-PINN (Bi, Deng, Zhu, 2026)

Description: For dynamic (time-varying) interfaces, adds a "level-set" variable as an extra input (independent variable) to the network and provides a dynamic solution.

✅ Pros: Provides high accuracy in time-dependent problems; works well with NKT (Network Theory) spectral bias; handles moving interfaces more accurately.

❌ Cons: Level-set function is complex (requires additional computation); can create unnecessary complexity in static problems or problems where the interface doesn't move much.

Boundary Conditions

Rules that specify what the solution should be at the outer boundary of the entire computational domain (at the edge of the domain, not at the interface). Two main types:

Dirichlet: The value itself is given at the boundary. Example: "Temperature at x=0 should be 310 K" -> u(0) = 310

Neumann: The derivative (flux/flow) is given at the boundary. Example: "Heat flux at x=L should be h" -> $-k \cdot du/dx = h$

These are necessary to define the problem — a PDE alone can have infinitely many solutions, and boundary conditions reduce this to a single solution.